

Acessar

[arineto1.github.io/ifpe2026](https://arineto1.github.io/ifpe2026)

# Informática Teórica

## Revisão - Aula 1

O que é Informática Teórica?

## Definição Técnica

A **Informática Teórica** é o estudo dos fundamentos matemáticos que definem **o que pode ser computado e com que eficiência**.

Segundo Sipser, ela se divide em **três áreas principais**:

- Teoria de Autômatos
- Teoria da Computabilidade
- Teoria da Complexidade





## AUTÔMATOS

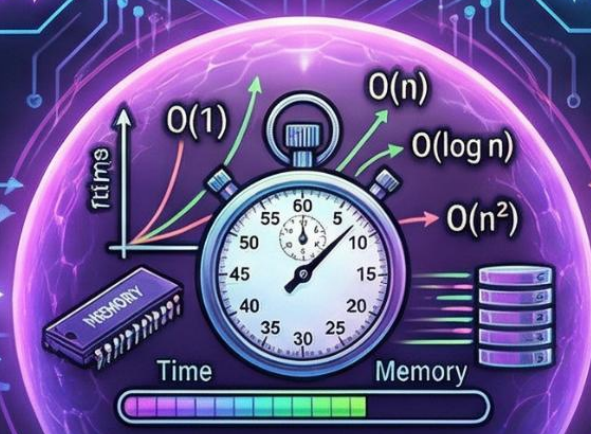
Como as máquinas processam?  
(Modelos e gramáticas)

$$\Sigma = \sum \text{Turing} \rightarrow \boxed{\text{Halt}} f(x)$$



## COMPUTABILIDADE

O que pode ser resolvido?  
(Limites e decidibilidade)



## COMPLEXIDADE

O quão rápido?  
(Recursos de tempo e memória)

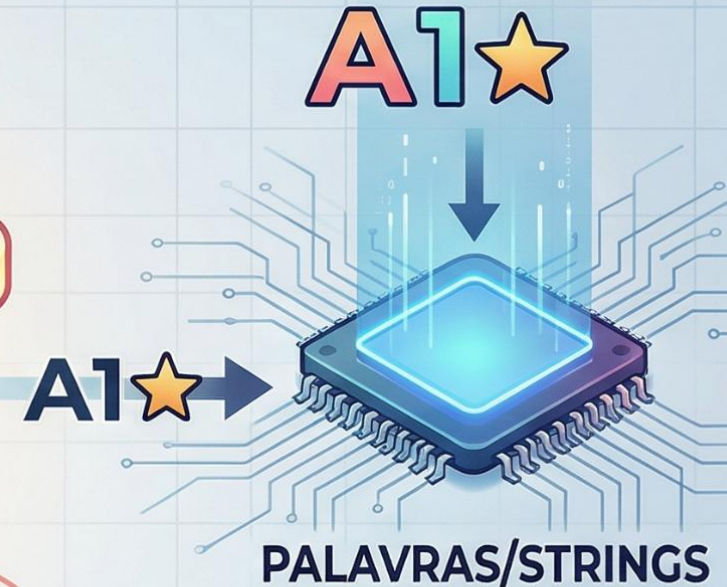
```
fast algorithm
: data_ap
: process_data
: proces_data
: read pr: )
```



O Alfabeto ( $\Sigma$ )

Um alfabeto é um **conjunto finito e não vazio** de símbolos.

## INFORMÁTICA TEÓRICA: DEFINIÇÃO DO ALFABETO ( $\Sigma$ )



NÃO PODE SER VAZIO

$$\Sigma \neq \emptyset$$



NÃO PODE SER INFINITO

$$|\Sigma| < \infty$$



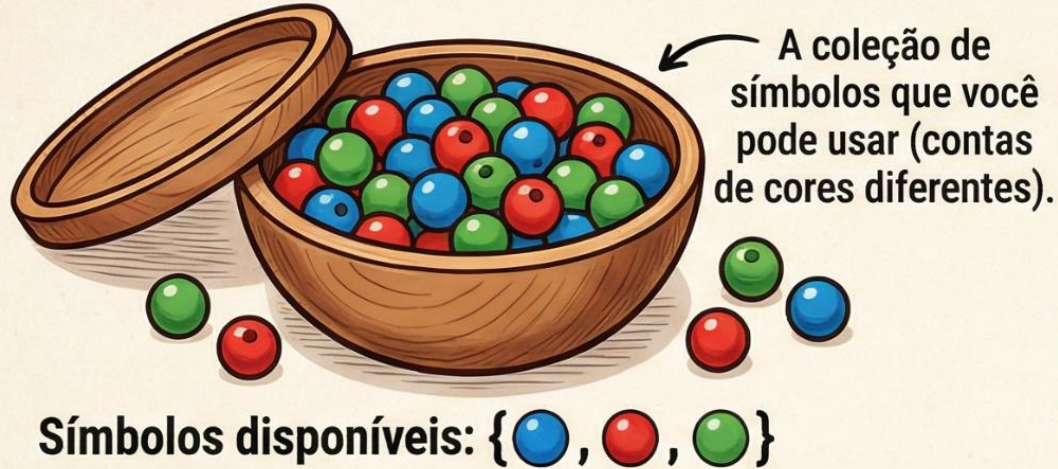
$|\Sigma| = 12$  distinct symbols

Cadeias ou Strings (w - word)

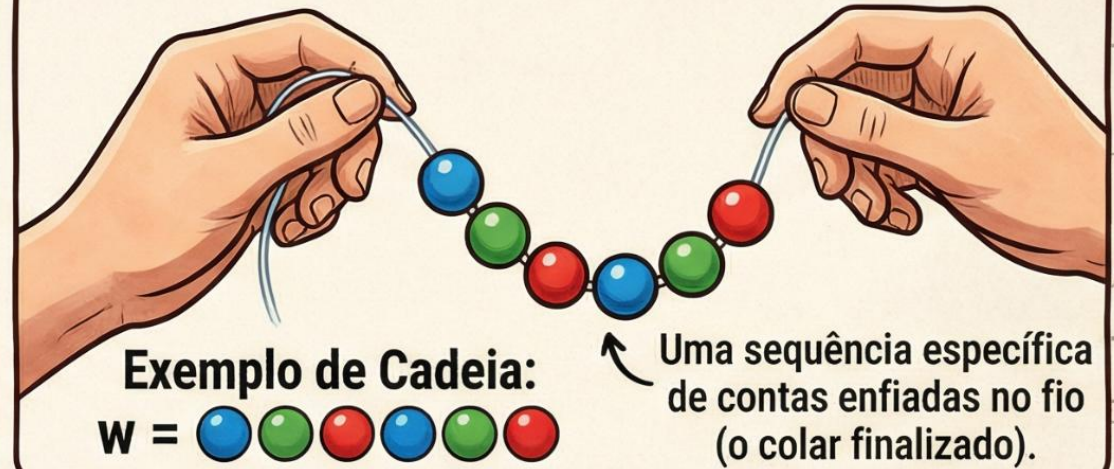


# Analogia do Colar de Contas: Informática Teórica

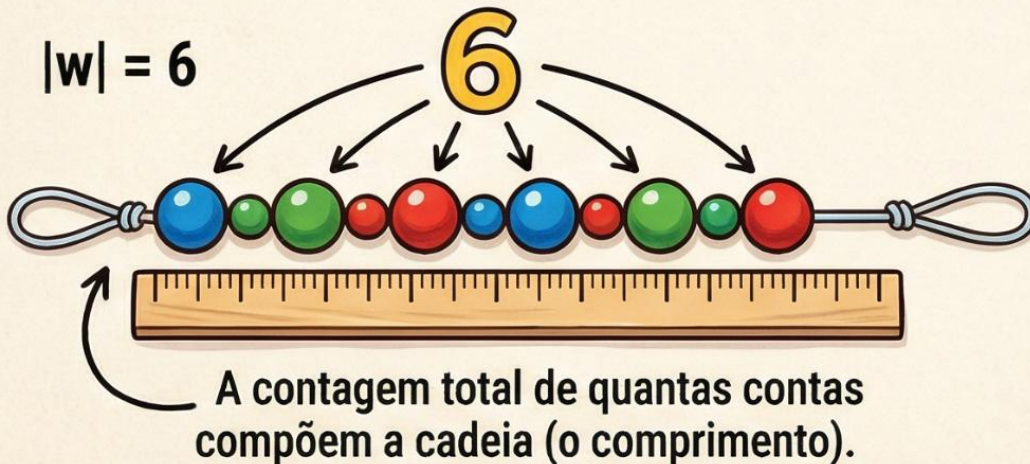
## 1 O Alfabeto ( $\Sigma$ ): Tipos de Contas Soltas



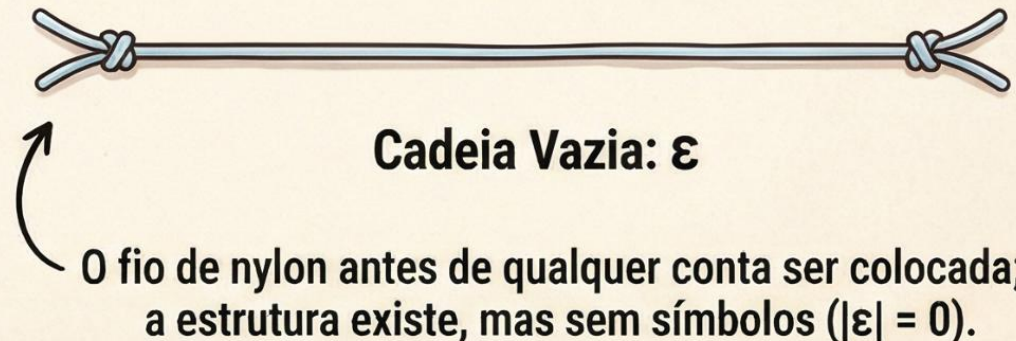
## 2 Uma Cadeia ( $w$ ): O Colar Montado



## 3 O Comprimento ( $|w|$ ): Contando as Contas



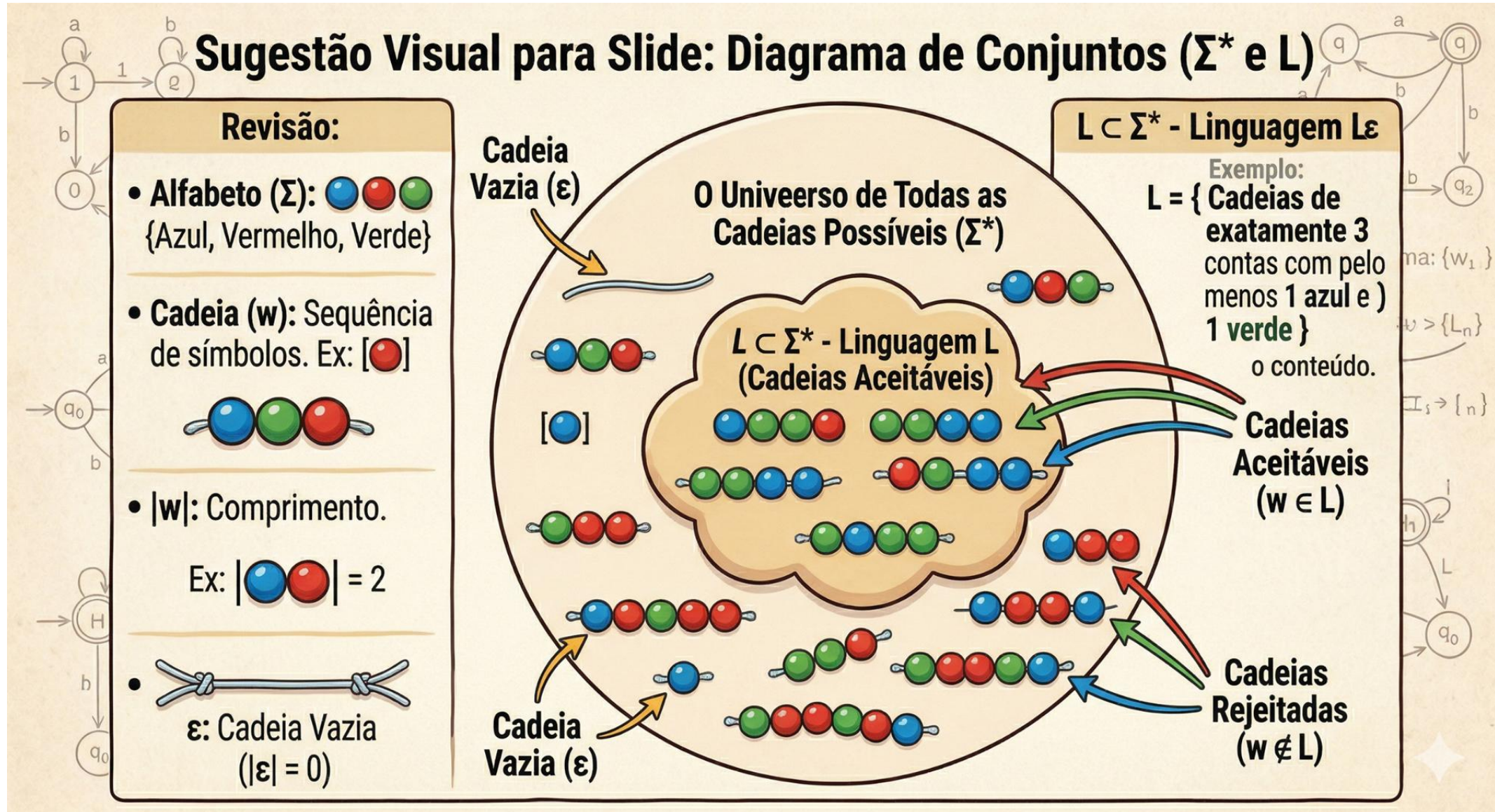
## 4 A Cadeia Vazia ( $\epsilon$ ): Apenas o Fio



Linguagens (L)



Uma linguagem é definida como um **conjunto de cadeias (strings)** escolhidas a partir de um **alfabeto  $\Sigma$** .

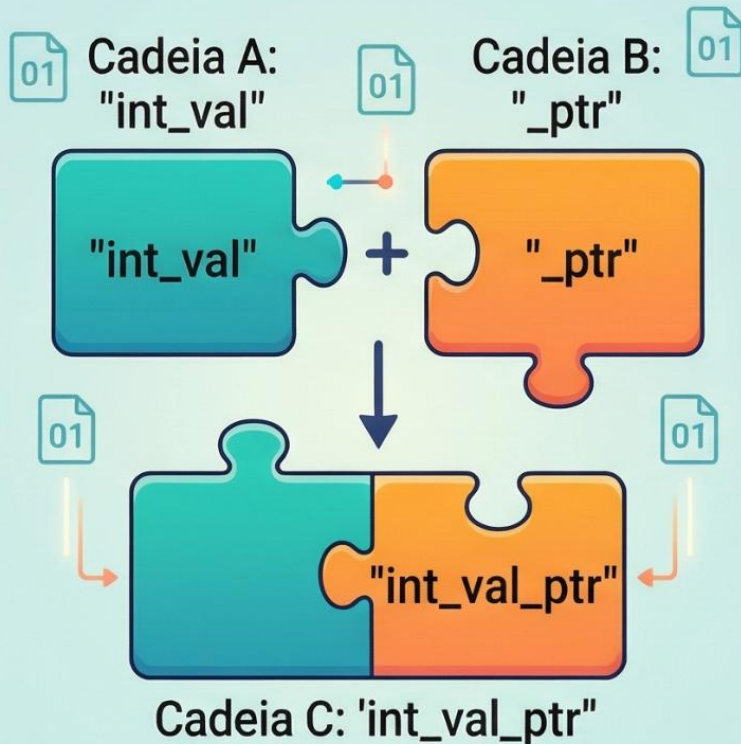


# Operações com Cadeias (w)



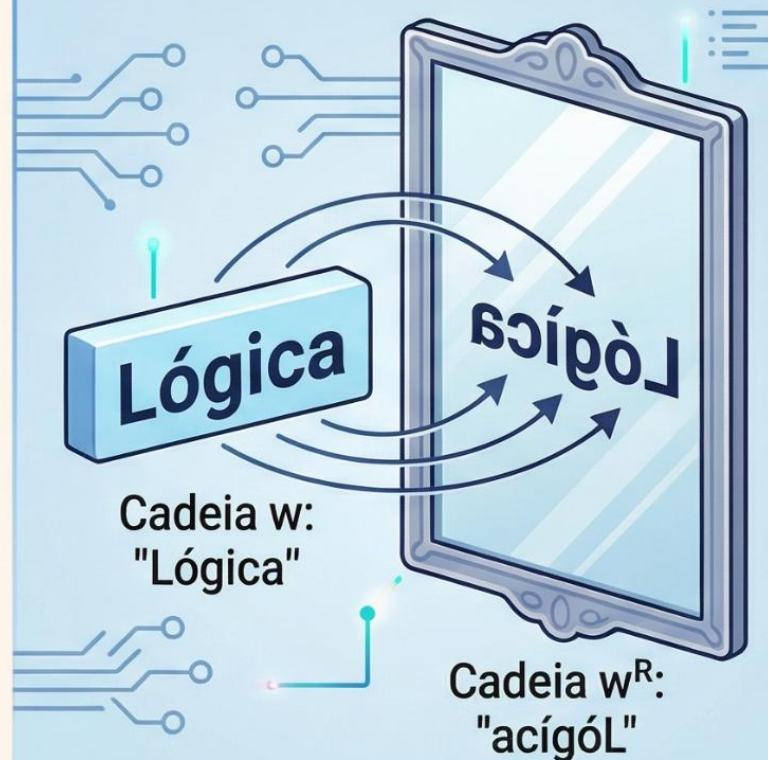
# CONCEITOS BÁSICOS DE LINGUAGENS FORMAIS (INFORMÁTICA TEÓRICA)

## 1. CONCATENAÇÃO ( $A \cdot B$ ): Encaixando Cadeias



Resultado: A união física de A e B, mantendo a ordem.  
Ex:  $C = A \cdot B$

## 2. REVERSA ( $w^R$ ): Inversão de Cadeia ( $w^R$ )



Inverte a ordem de todos os símbolos da cadeia.  
Ex:  $w^R$

## 3. POTÊNCIA ( $w^n$ ): Repetição de Cadeia ( $w^n$ )

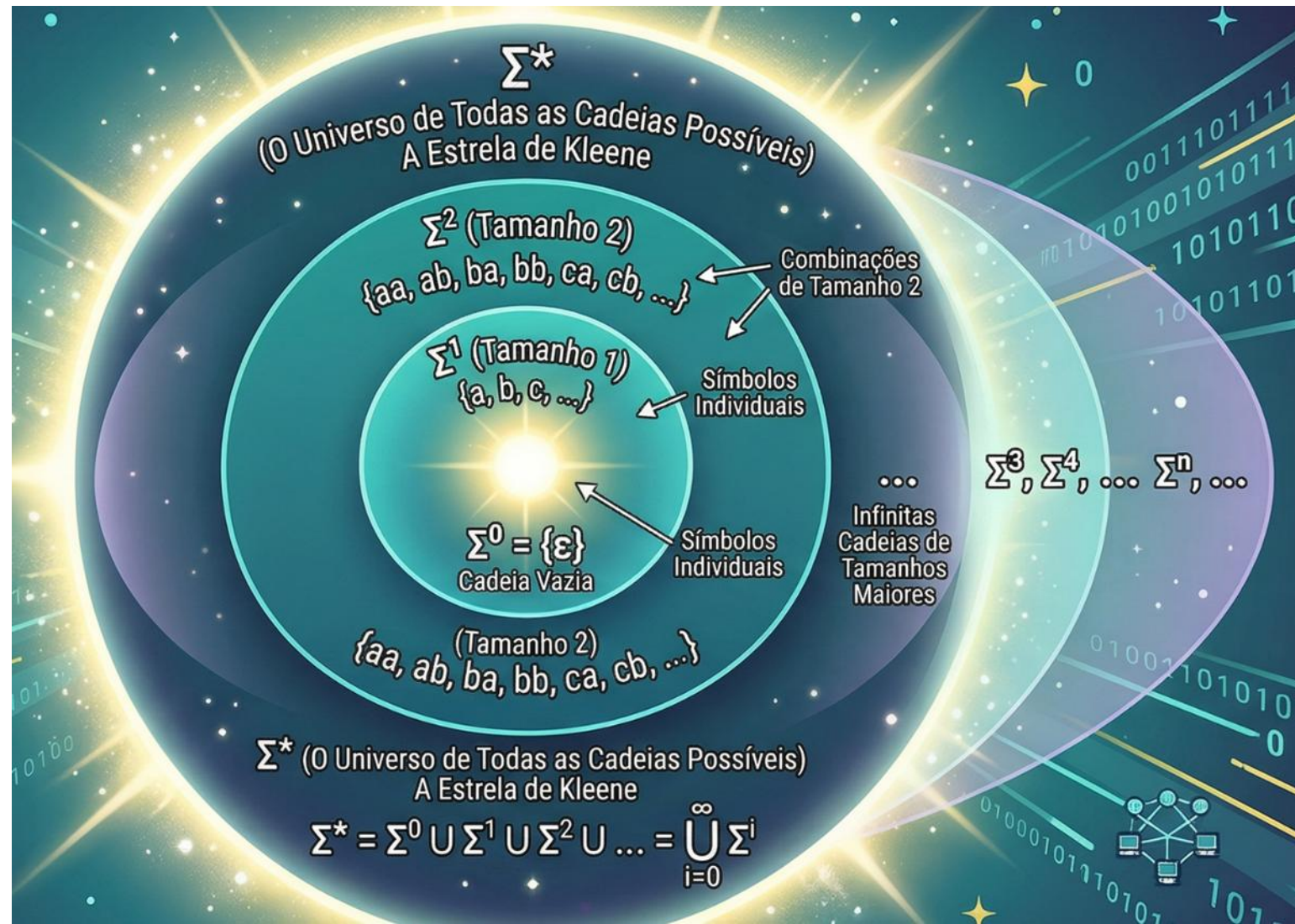


Repete a cadeia "w" "n" vezes consecutivamente.  
Ex:  $w^3 = 010101$

O Fecho de Kleene ( $\Sigma^*$ )



O Fecho de Kleene (representado pelo símbolo de estrela  $*$ ) de um alfabeto  $\Sigma$  é o conjunto de todas as cadeias possíveis que podem ser geradas a partir dos símbolos desse alfabeto, incluindo a cadeia vazia ( $\epsilon$ ).





# Informática Teórica

## Aula 2

# **Introdução aos Autômatos Finitos Determinísticos (AFD)**

## Definição Técnica

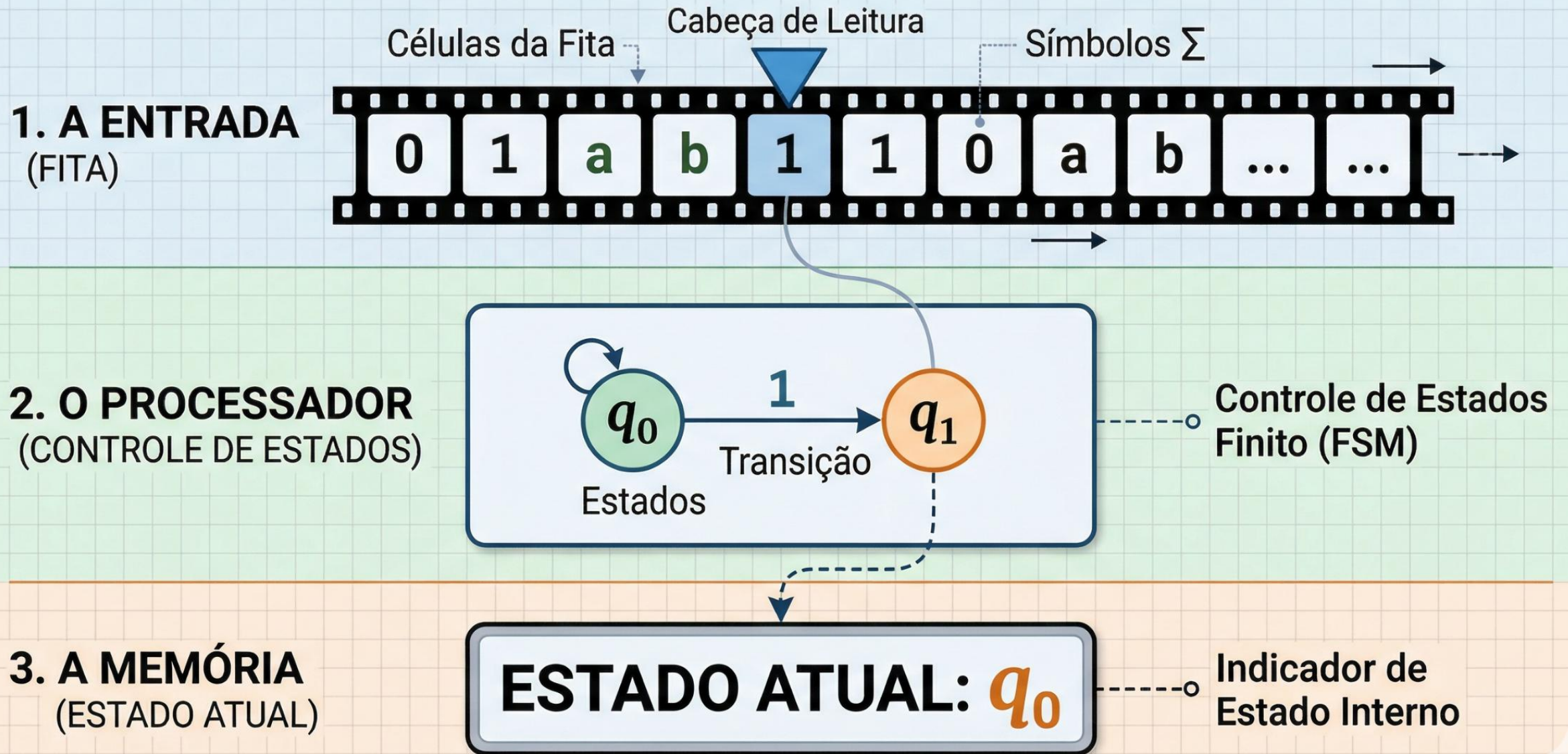
O **Autômato Finito** é o modelo de computação **mais simples e fundamental** na Teoria da Computação.

Segundo Sipser (2007), estes modelos são ideais **para computadores com uma quantidade extremamente pequena de memória**, focando-se no reconhecimento de padrões em sequências de entrada.

Hopcroft (2001) descreve-os como sistemas que se movem através de um **conjunto finito de estados em resposta a estímulos externos** (símbolos de um alfabeto).



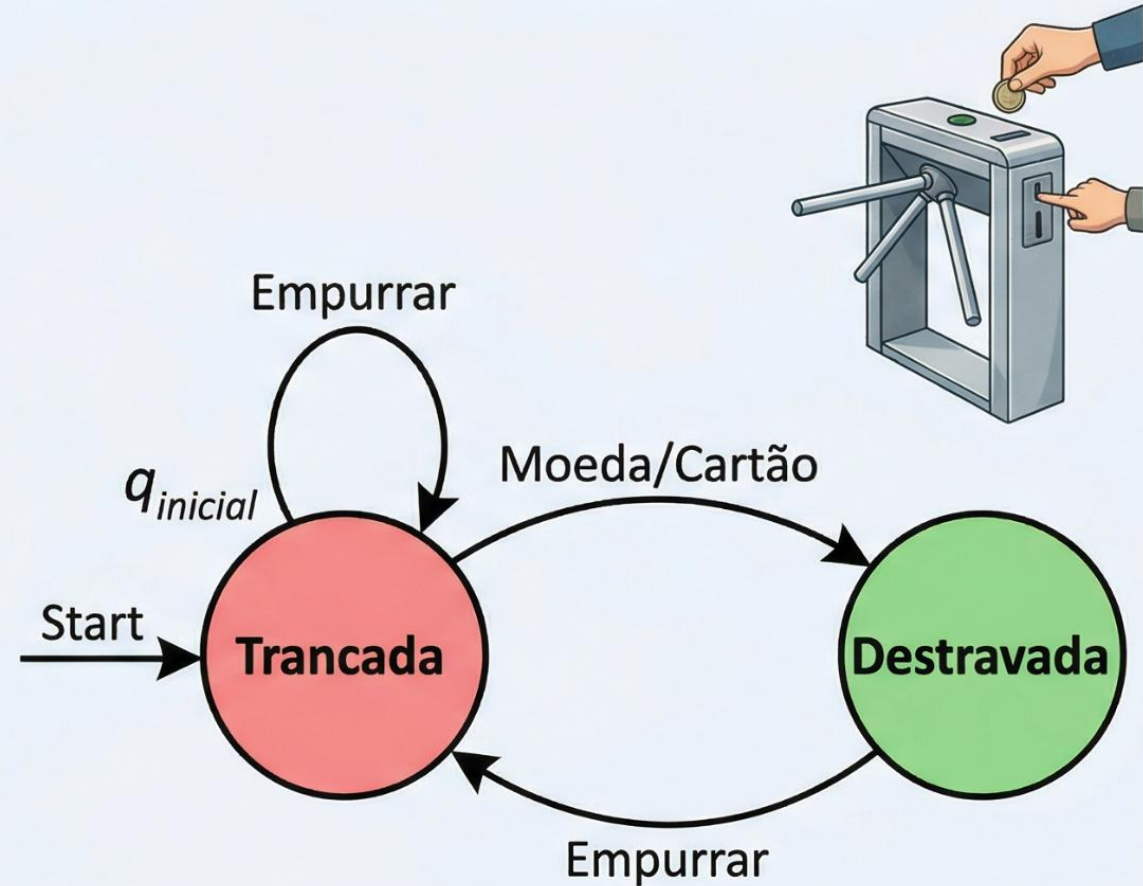
# Informática Teórica: O Fluxo da Computação (Modelo DFA)



# Analogia: A Catraca de Acesso

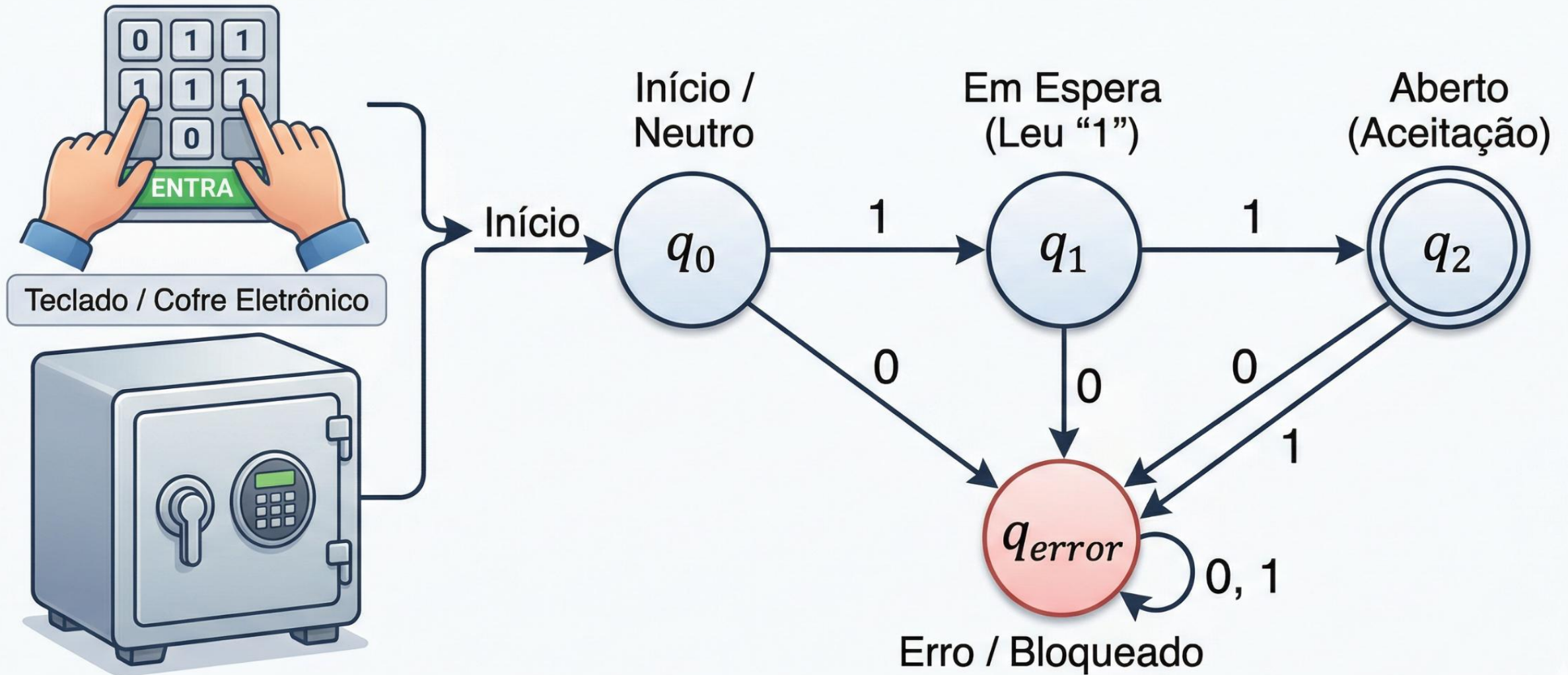
## Modelando o comportamento como um Autômato Finito

- **Estados:** Ela está “Trancada” (inicial) ou “Destravada”.
- **Entradas:** “Inserir Moeda/Cartão” ou “Empurrar o Braço”.
- **Lógica:**
  - Se está Trancada e você Empurra, ela continua Trancada.
  - Se está Trancada e você Insere Moeda, ela passa para Destravada.
  - Se está Destravada e você Empurra, ela volta a ficar Trancada.
- *Nota final:* Nesta analogia, o autômato “controla” o acesso baseado nas ações recebidas.

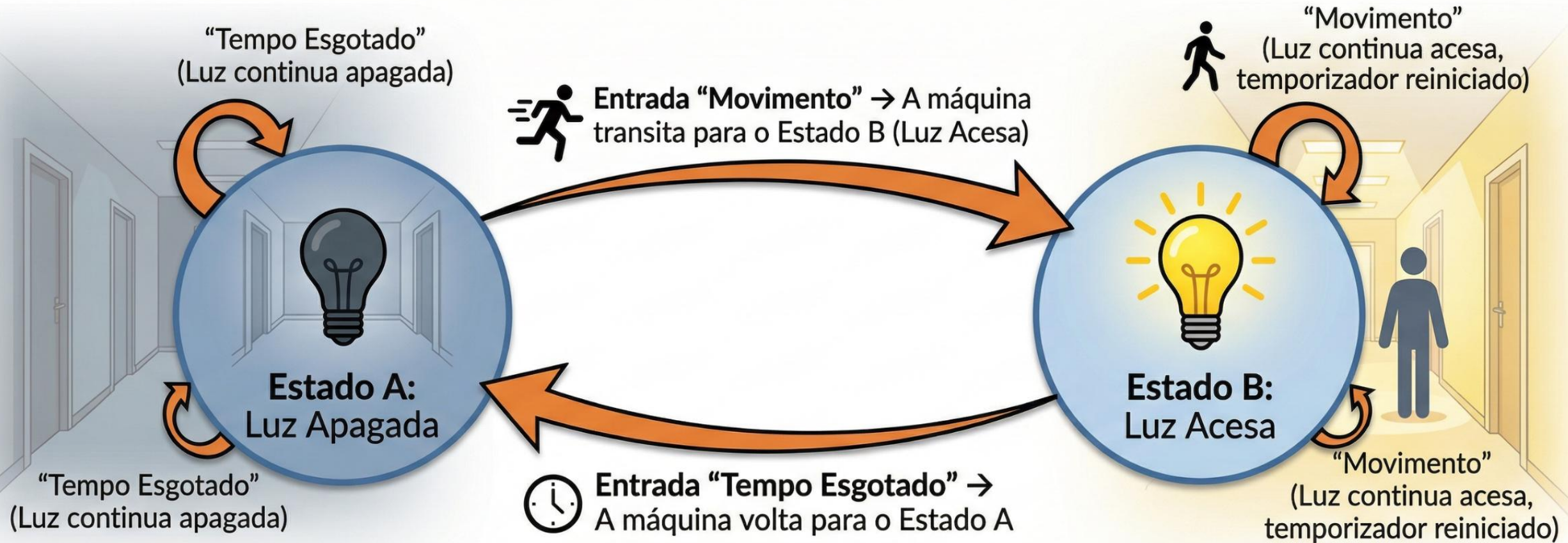




# Exemplo Prático: Reconhecedor de Senha “11”



# Exemplo Prático: Sensor de Presença



## Lógica: É Determinístico

Determinístico porque o sensor sempre reage da mesma forma ao mesmo estímulo, dependendo apenas se a luz já está acesa ou não.



## **A Definição Formal (A 5-upla) - Quintupla**

## 1. Definição Técnica Detalhada

Um Autômato Finito Determinístico é um modelo matemático representado por uma **5-upla M (Eme)** = (**Q (Quê)**,  **$\Sigma$  (Sigma)**,  **$\delta$  (Delta)**,  **$q_0$  (Quê zero)**, **F (Efe)**). Cada componente possui uma função específica:

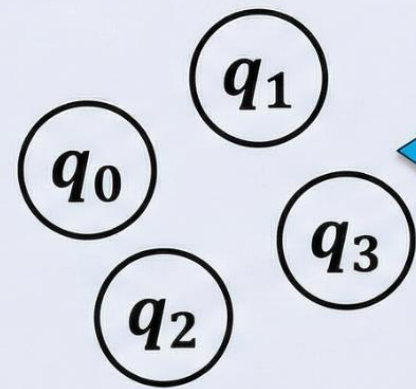
- **Q (Quê maiúsculo)**: É o conjunto finito de **estados**. Representa todas as "situações" ou memórias que a máquina pode assumir.
- **$\Sigma$  (Sigma maiúsculo)**: É o **alfabeto**. Um conjunto finito de símbolos (como 0 e 1) que a máquina consegue ler.
- **$\delta$  (Delta minúsculo)**: É a **função de transição**. É a regra lógica que diz: "se estou neste estado e leio este símbolo, devo ir para aquele estado".
- **$q_0$  (Quê minúsculo com índice zero)**: É o **estado inicial**. Indica o ponto exato onde a máquina começa a trabalhar.
- **F (Efe maiúsculo)**: É o conjunto de **estados de aceitação** (ou finais). São os estados que, se a máquina parar neles ao fim da leitura, significam que a palavra foi aceita.

# O Mapa da Máquina

## Definição Formal de um Autômato Finito Determinístico (DFA)

**$Q$ : Conjunto de Estados**  
(Círculos Vazios)

**$\Sigma$ : Alfabeto de Entrada**  
(‘0’, ‘1’)

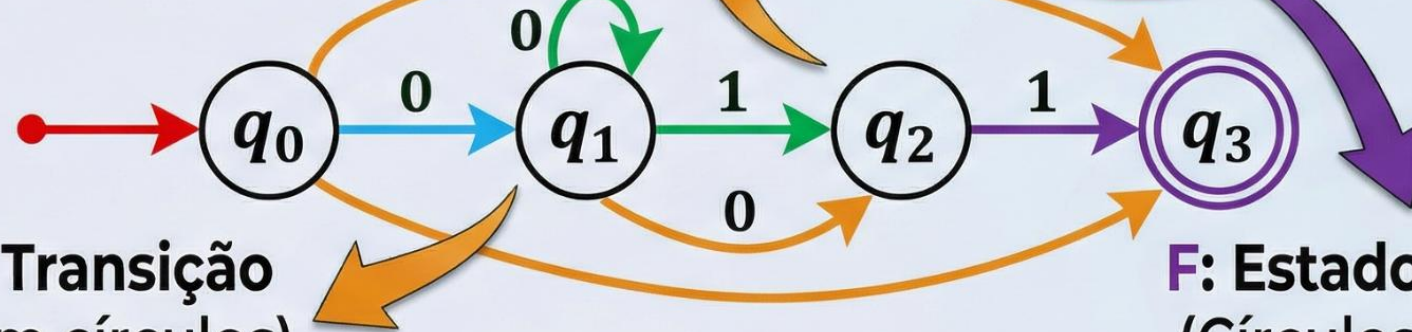


$$M = (Q, \Sigma, \delta, q_0, F)$$

**$q_0$ : Estado Inicial**  
(Seta sem origem)

**$\delta$ : Função de Transição**  
(Setas que ligam círculos)

**$F$ : Estados Finais/Aceitação**  
(Círculos com borda dupla)





# Analogia: O Painel de um Carro

**Q (Estados):** São as posições da marcha (P, R, N, D).



Gear alavanca



Ações do Motorista



Pisan Freio

$$M = (Q, \Sigma, \delta, q_0, F)$$



**$\Sigma$  (Alfabeto):** São as suas ações (Mudar a alavanca, Pisar no freio).

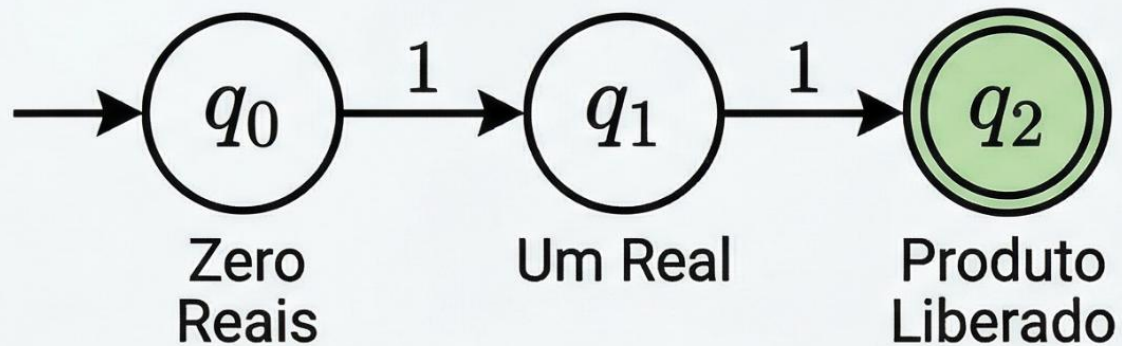
**$\delta$  (Função):** É a regra: “Se estou em P e movo a alavanca para baixo, vou para R”.

**$q_0$  (Início):** O carro sempre liga em P (Park).

**F (Aceitação):** O estado D (Drive), pois é o único que permite que o carro cumpra seu objetivo de andar.



## Exemplo Prático: Máquina de Refrigerante como Autômato Finito (AFD)



$Q = \{q_0, q_1, q_2\}$ : Estados

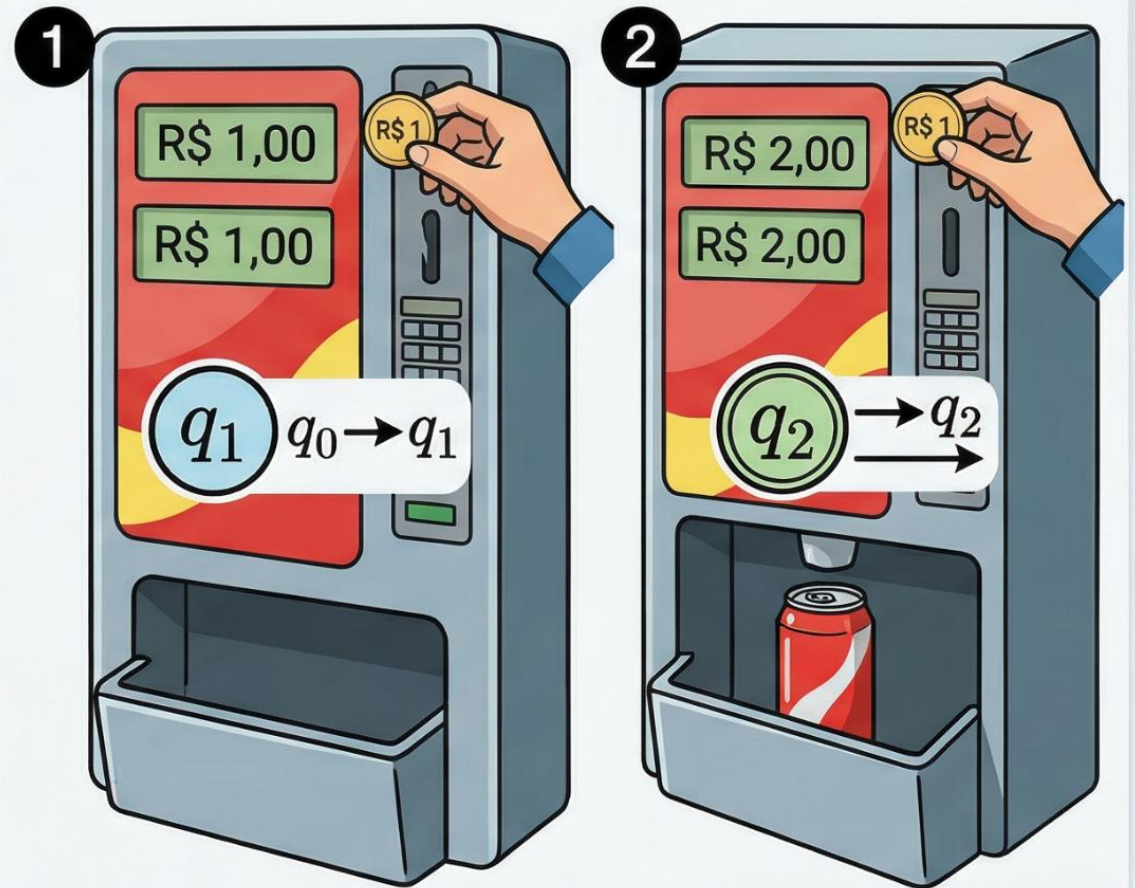
$\Sigma = \{1\}$ : Alfabeto (moeda de R\$ 1)

$q_0$ : Estado Inicial

$F = \{q_2\}$ : Estado Final

$\delta$ : Função de Transição       $\delta(q_0, 1) = q_1$

$\delta(q_1, 1) = q_2$



# **Diagramas de Transição (O Desenho da Máquina)**



## Definição Técnica

O diagrama de transição é uma **representação gráfica do autômato** em forma de grafo direcionado, onde cada componente da 5-upla assume uma forma visual:

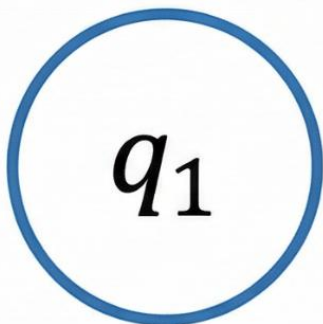
- **Nós ( $Q$  - Quê):** Os círculos representam os **estados** (Quê) da máquina.
- **Arestas ( $\delta$  - Delta):** As setas representam a **função de transição** (Delta), indicando o movimento entre os estados.
- **Rótulos ( $\Sigma$  - Sigma):** Os símbolos do **alfabeto** (Sigma) sobre as setas indicam qual caractere ativa aquela transição.
- **Seta de Entrada ( $q_0$  - Quê zero):** Uma seta "solta" que aponta para o **estado inicial** (Quê zero).
- **Círculo Duplo ( $F$  - Efe):** Identifica os estados que pertencem ao conjunto de **estados de aceitação** (Efe).



# O Alfabeto Visual



Estado (Q - Quê)



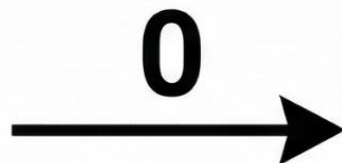
Um círculo simples =  
**Estado (Q - Quê)**



Define todos os  
estados possíveis  
(ex:  $q_1, q_2$ ).



Transição ( $\delta$  - Delta)



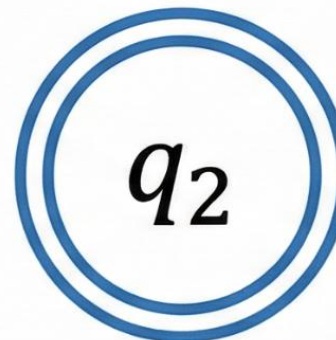
Uma seta com um '0'  
em cima = **Transição ( $\delta$  -  
Delta)** por meio do  
símbolo ( $\Sigma$  - Sigma)



Muda de estado com  
base na entrada (ex:  
de  $q_0$  para  $q_1$  ao ler 0)



Estado Final (F - Efe)



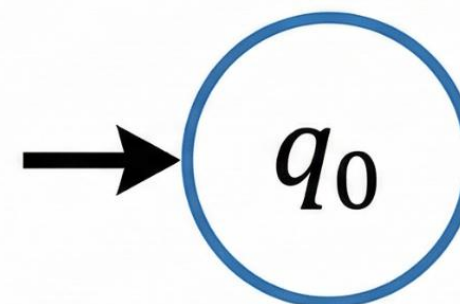
Um círculo com dois  
contornos = **Estado  
Final (F - Efe)**



Estado onde o autômato  
'aceita' a cadeia (ex:  
 $F = \{q_2\}$ ).



Início ( $q_0$  - Quê zero)



Uma seta curta  
apontando para o  
primeiro círculo  
= **Início ( $q_0$  - Quê zero)**

Indica onde o  
processamento começa  
(ex:  $q_0$ ).



# Autômato Finito Determinístico (DFA)

## 1. Unicidade (O caminho é único)

Para cada estado e para cada símbolo do alfabeto, deve existir **exatamente uma** transição de saída.

## 2. Totalidade (Sem "buracos")

A função de transição deve ser total. Isso significa que **todo símbolo do alfabeto deve ter um destino definido** em todos os estados.

## 3. Proibição de Transições Vazias ( $\epsilon$ )

Um DFA **nunca** pode mudar de estado sem consumir um caractere da entrada.



## 4. Alfabeto e Estados Finitos

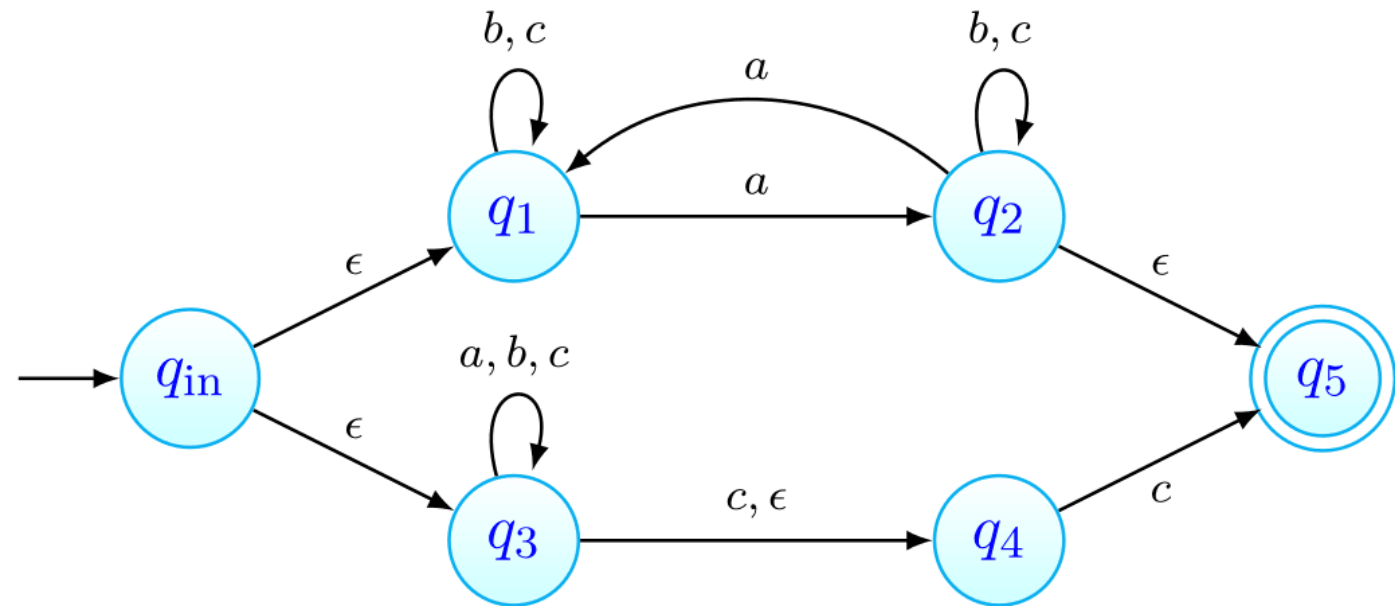
Como o nome diz, tudo deve ser finito:

- O conjunto de estados ( $Q$ ) deve ser finito.
- O alfabeto ( $\Sigma$ ) deve ser um conjunto finito de símbolos.

## 5. Um Único Estado Inicial

Um DFA começa sempre em um único lugar. Você não pode ter duas setas de "entrada" apontando para estados diferentes no início do processo.

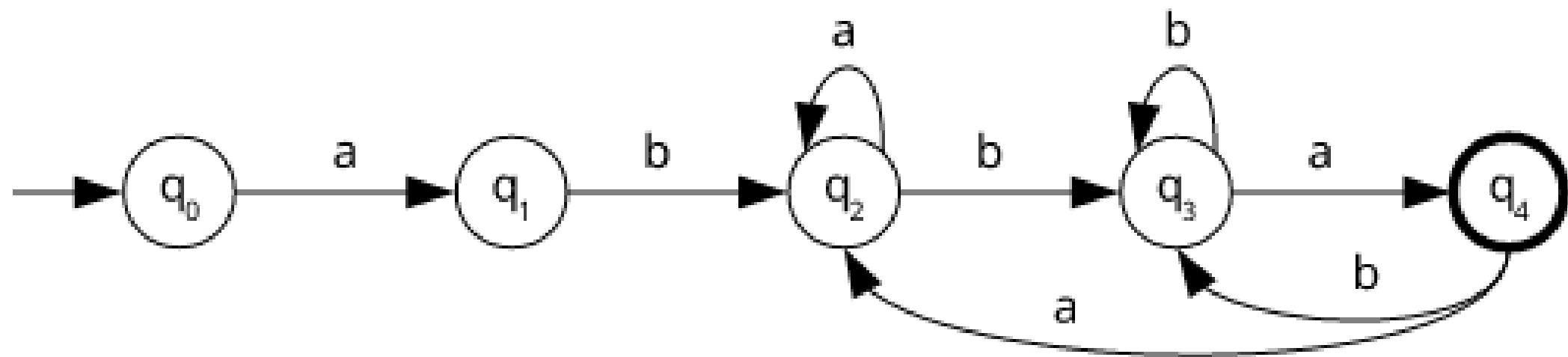
Não, esse autômato **não é um Autômato Finito Determinístico (DFA)**. Ele é um **Autômato Finito Não-Determinístico com transições vazias ( $\epsilon$ -NFA)**.



**1. Presença de transições  $\epsilon$  (Epsilon)**

**2. Não-determinismo (Múltiplas escolhas)**

Em um DFA, para cada estado e cada símbolo do alfabeto, deve haver **exatamente uma** transição saindo. Nesse grafo, há ambiguidade:



**Por que ele não passou no teste?**

Para um autômato ser formalmente determinístico, ele precisa ter uma resposta para **cada letra do alfabeto em cada estado**. Observe onde ele falha:

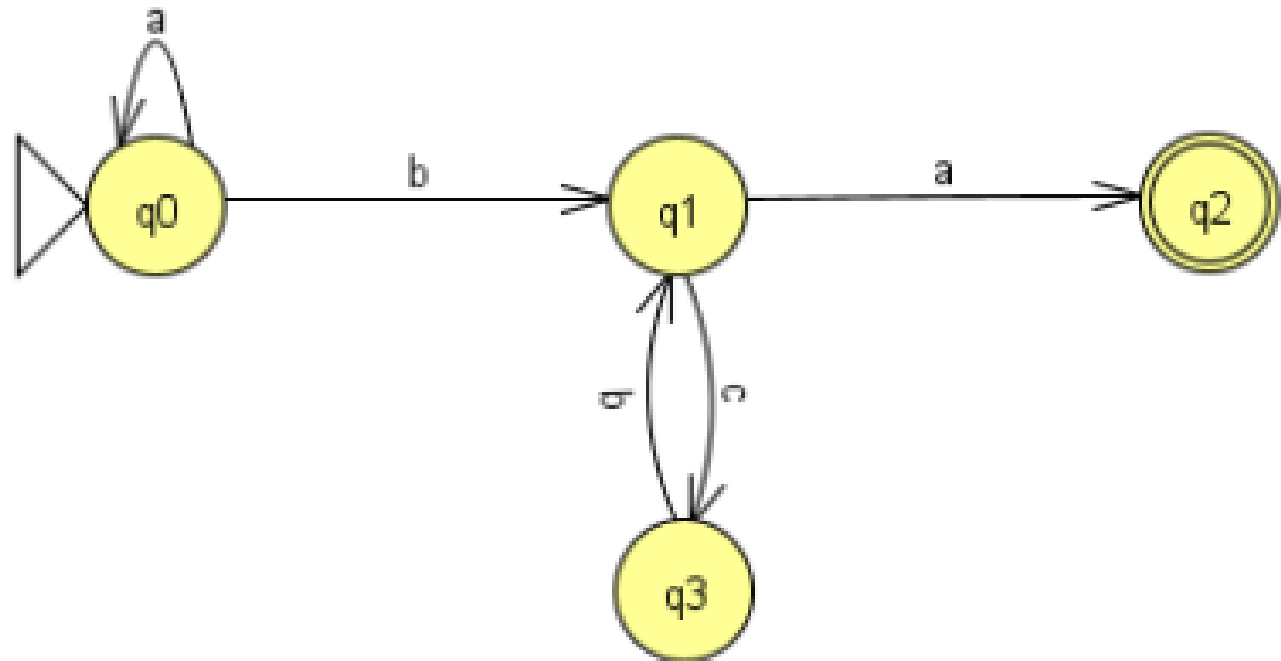
- **Estado  $q_0$ :** Só existe transição para a letra '**a**'. Se a entrada começar com '**b**', o autômato não tem instrução de para onde ir.
- **Estado  $q_1$ :** Só existe transição para a letra '**b**'. Se chegar um '**a**' nesse estado, ele "trava".



Ainda não! Este também é um **Autômato Finito Não-Determinístico (AFN)**.

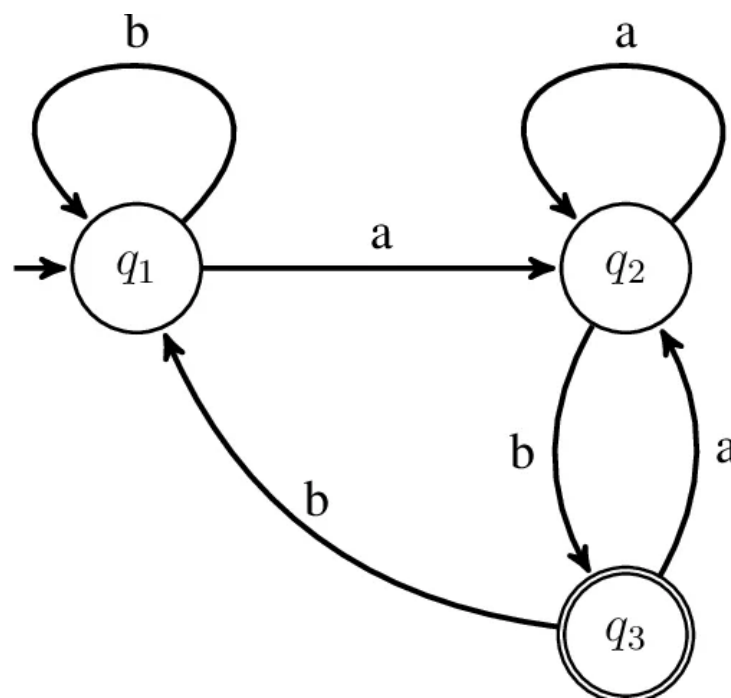
Embora ele não tenha aquelas transições vazias ( $\epsilon$ ) que vimos no primeiro exemplo, ele falha no critério da **completude**, assim como o segundo gráfico que você enviou.

Para ser um **DFA**, cada estado deve ter uma "saída" para cada letra do alfabeto. Considerando que o alfabeto aqui é  $\{a, b, c\}$ , veja por que ele não é determinístico:



## Por que este é um DFA?

1. **Completude (Totalidade):** Cada estado ( $q_1, q_2, q_3$ ) possui exatamente uma transição de saída para **cada símbolo** do alfabeto ( $a$  e  $b$ ). Não há "becos sem saída".
2. **Unicidade:** Não há ambiguidade. Ao ler um 'a' no estado  $q_1$ , você só tem um destino possível ( $q_2$ ). Não existem duas setas com a mesma letra saindo do mesmo lugar para destinos diferentes.
3. **Ausência de  $\epsilon$ :** Não existem transições vazias ou espontâneas. O autômato só se move se ler um caractere.





# O Mapa do Metrô (Como um Autômato Finito)



- ESTAÇÕES = ESTADOS ( $Q$ )
- TRILHOS = TRANSIÇÕES ( $\delta$ )
- BILHETES DOS PASSAGEIROS = SÍMBOLOS DO ALFABETO ( $\Sigma$ )
- ESTAÇÃO FINAL COM SAÍDA = ACEITAÇÃO ( $F$ )



Início  
( $q_0$ )

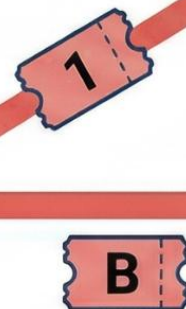
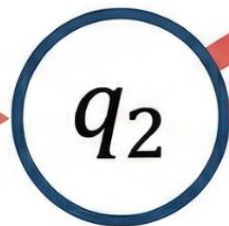
(Embarque)



(Parque)



(Comércio)



$q_F$

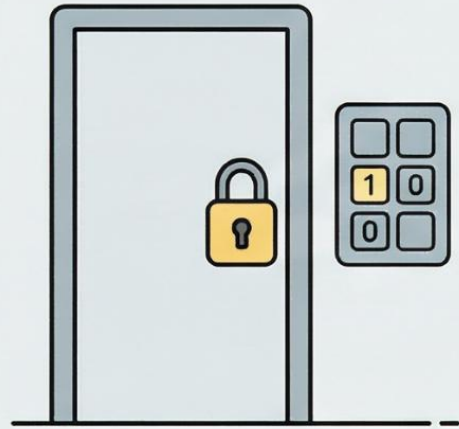
SAÍDA LIBERADA  
PARA A CIDADE

SAÍDA LIBERADA  
PARA A CIDADE





## 1. O Problema



$w = 10 \rightarrow \text{ABRE}$

Você quer projetar uma fechadura digital que só abre se a pessoa digitar exatamente a sequência “10”.

outra combinação mantém a porta travada.

## 2. A Modelagem (A 5-upla na prática)

$\Sigma$  (Sigma):  $\{0, 1\}$  (Os botões disponíveis no teclado)

$Q$  (Quê):  $\{q_0, q_1, q_2\}$

$q_0$  (Quê zero): Porta Travada (Esperando início)

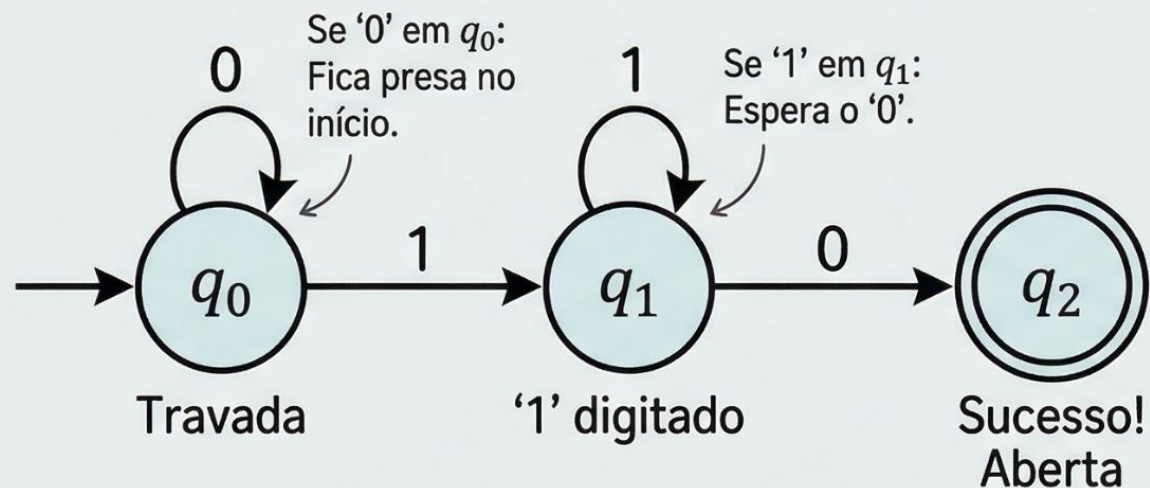
$q_1$  (Quê um): Primeiro dígito correto ('1') digitado

$q_2$  (Quê dois): Sucesso! Porta Aberta

$F$  (Efe):  $\{q_2\}$  (O único estado onde a porta abre)

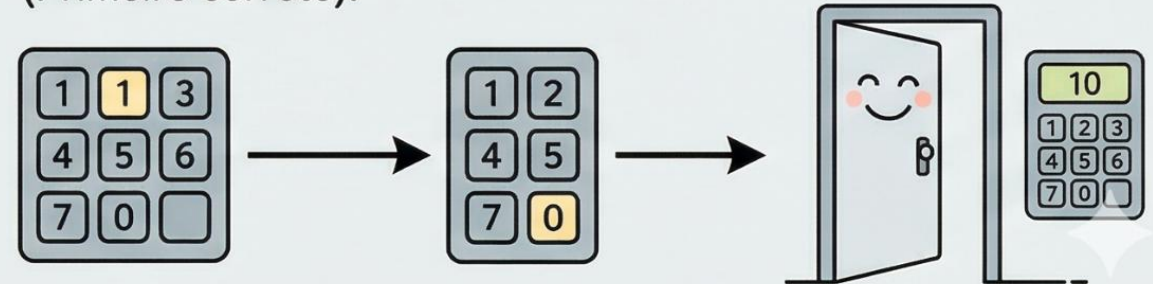
$\delta$ : Função de transição (mostrada no diagrama)

## 3. Sugestão Visual: O Diagrama de Estados



## 4. Simulação de Uso (Rastreamento)

- 1 Entrada:  $w = 10$ . Lê '1': A  $\delta$  diz para ir para  $q_1$  (Primeiro correto).
- 2 Lê '0': A  $\delta$  diz para ir para  $q_2$  (Porta abre).
- 3 Fim da Cadeia: A máquina parou em  $q_2$ ? Sim! Então a porta abre (Aceitação).



## **Processamento - Aceitação vs. Rejeição**

Como o **autômato** decide se uma **sequência de símbolos** é **correta ou não**

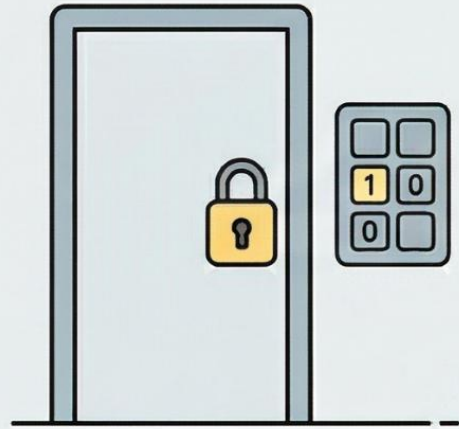
## 1. Definição Técnica

O processamento de uma cadeia  $w$  (**dáblío**) ocorre através da aplicação sucessiva da  $\delta$  (**Delta**), partindo de  $q_0$  (**Quê zero**).

- **Aceitação:** Segundo Sipser , uma cadeia é aceita se, após ler todos os símbolos, a máquina para em um estado que pertence ao conjunto **F (Efe)**.
- **Rejeição:** Ocorre quando a leitura termina em um estado que **não** pertence a **F (Efe)** ou quando a máquina entra em um "Estado de Erro" (ou Trap State).
- **Determinismo:** De acordo com Hopcroft , para cada par de estado e símbolo, deve haver exatamente uma transição definida. Se o símbolo lido não levar a um caminho de sucesso, a máquina deve "travar" ou ir para um estado de descarte.



## 1. O Fluxograma de Decisão



$w = 10 \rightarrow$  ABRE

Você quer projetar uma fechadura digital que só abre se a pessoa digitar exatamente a sequência "10".

outra combinação mantém a porta travada.

## 2. A Modelagem (A 5-upla na prática)

$\Sigma$  (Sigma):  $\{0, 1\}$  (Os botões disponíveis no teclado)

$Q$  (Quê):  $\{q_0, q_1, q_2, q_{\text{erro}}\}$

$q_0$  (Quê zero): Porta Travada (Esperando início)

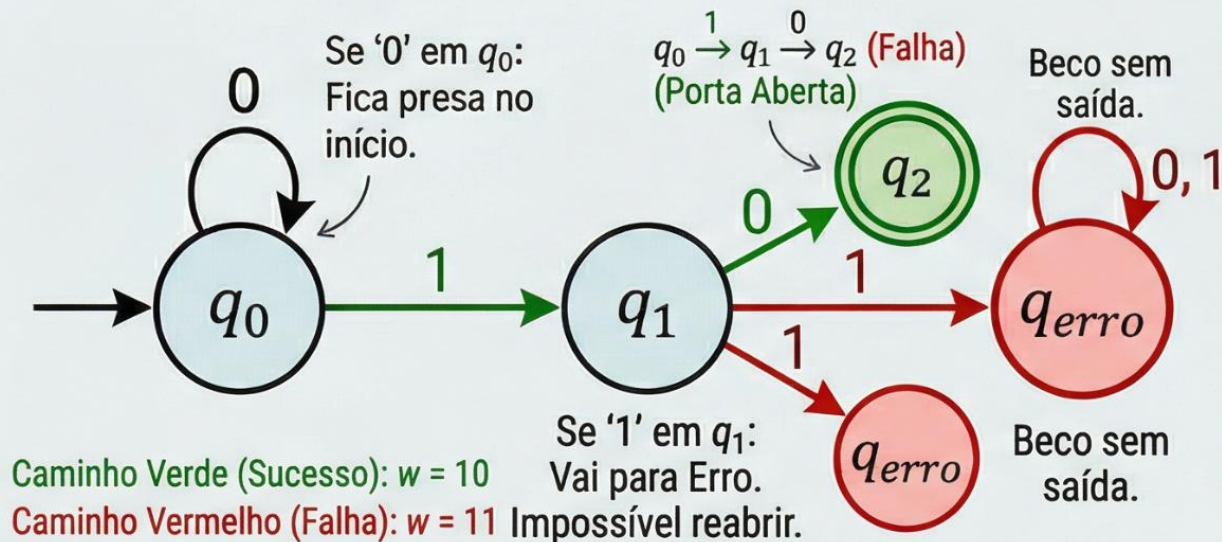
$q_1$  (Quê um): Primeiro dígito correto ('1') digitado

$q_2$  (Quê dois): Sucesso! Porta Aberta

$q_{\text{erro}}$ : Caminho de Falha Irreversível

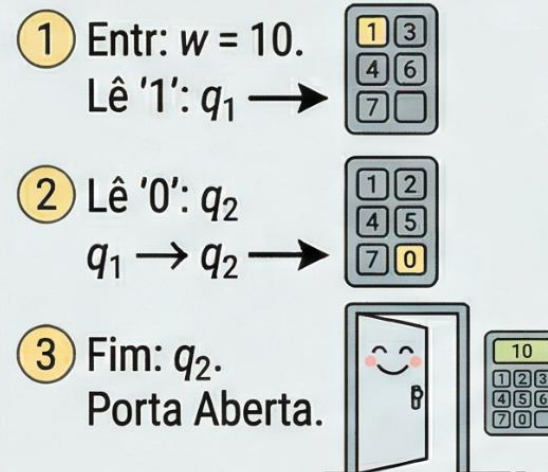
$\delta$ : Função de transição (mostrada no diagrama)

## 3. Sugestão Visual: O Diagrama de Estados



## 4. Simulação de Uso (Rastreamento)

Rastreamento  $w = 10$  (Sucesso)



Rastreamento  $w = 11$  (Falha)



## Linguagens Regulares e a Hierarquia

Conectando os **AFDs** ao conceito de **Linguagens Regulares**. Este conhecimento se encaixa no **mapa completo da computação**, preparando o terreno para as próximas aulas sobre **linguagens mais complexas**.

## Definição Técnica

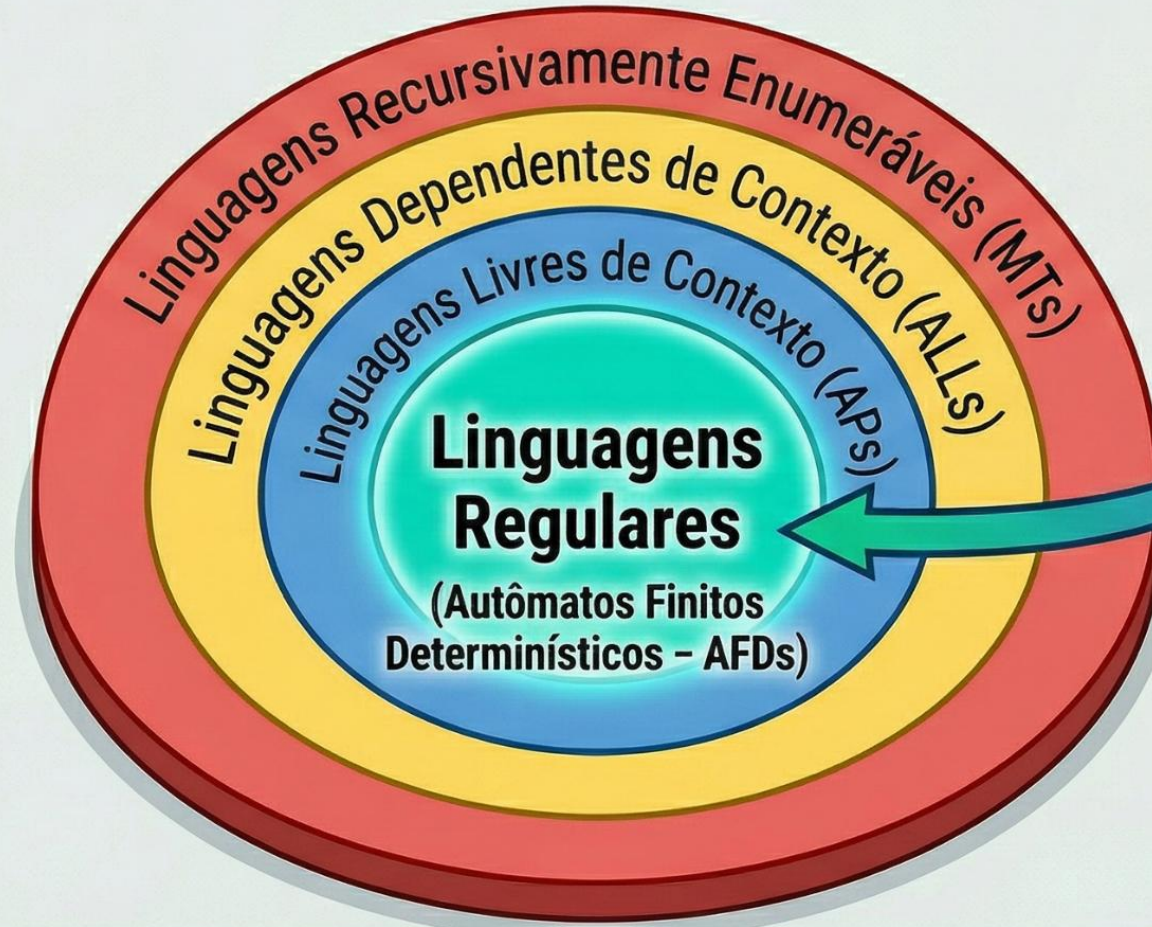
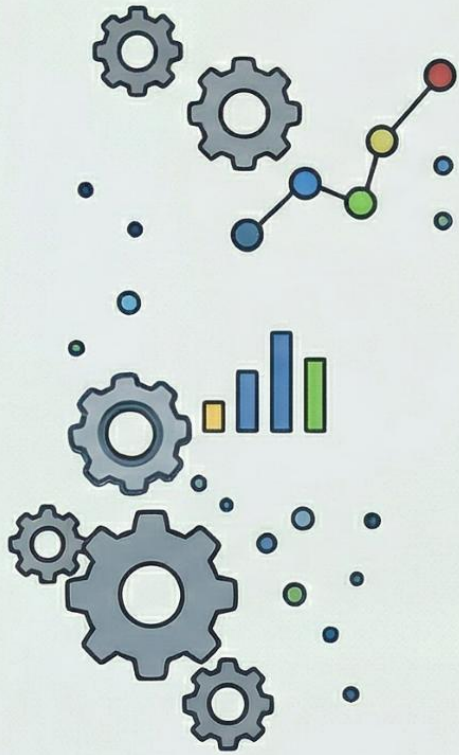
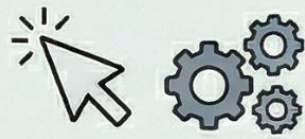
Uma linguagem é classificada como **Regular** se, e somente se, existir algum **Autômato Finito** que a reconheça.

**Segundo Sipser**, o conjunto de todas as linguagens que podem ser decididas por **AFDs** forma a classe das **Linguagens Regulares**.

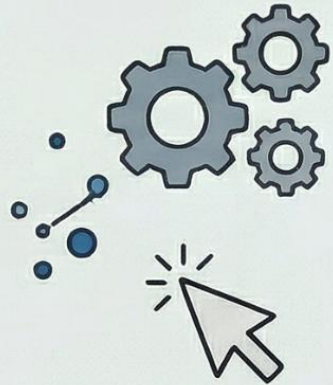
**Hopcroft** destaca que estas linguagens estão no nível mais básico da **Hierarquia de Chomsky** (Nível 3), sendo fundamentais para a construção de compiladores e analisadores léxicos.



# A Hierarquia de Chomsky (Modelos de Linguagem)



**Hoje: Dominando a BASE de Tudo!**  
(Simplicidade = Poder)



**Legenda do Círculo Interno: O que define a Simplicidade?**

- ①  $q_0$  **Q (Estados):** Finitos, pré-definidos. Sem memória extra.    ②   **$\Sigma$  (Alfabeto):** Símbolos de entrada imutáveis.    ③  $\rightarrow$   **$\delta$  (Delta - Transição):** Regras operacionais fixas e simples.



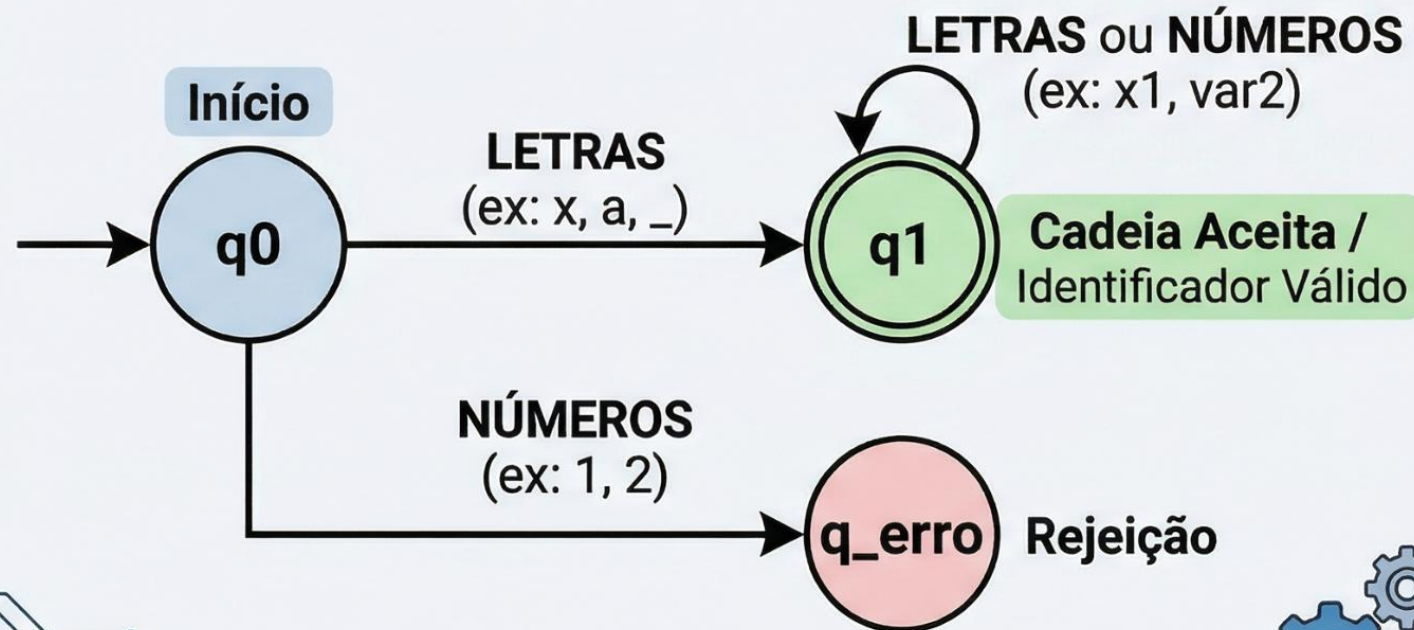
# Exemplo Prático: Identificadores de Variáveis

## Regra de Nomes de Variáveis

Um nome de variável **DEVE** começar com uma **LETRA** e **PODE** ser seguido por **LETRAS** ou **NÚMEROS**.

## Autômato Finito Determinístico (AFD) para Validação

É Regular? Sim! Um AFD simples de 2 estados valida isso.



## Por que Importa? O Papel do Compilador

```
code.cc
1 #include <stdio.h>
2
3 int main() {
4     int x = 10;
5     if (x == 1) {
6         printf("Valor 1");
7     }
8 }
9
10 else {
11     int y = 20;
12     if (y > 0) {
13         printf("Valor 2");
14     }
15 }
```

✅ **Válido**  
(Identificador Correto)

`int x1 = 10;`

`int 1x = 20;`

**Compilador**  
(Análise Sintática)

❌ **Erro de Sintaxe:**  
**Identificador Inválido**  
(começa com número)

O compilador usa essa lógica exata de autômatos para **validar os nomes** das **suas variáveis**.

Acessar

[arineto1.github.io/ifpe2026](https://arineto1.github.io/ifpe2026)